Analysis of Massive Data Sets

http://www.fer.hr/predmet/avsp

Prof. dr. sc. Siniša Srbljić

Izv. prof. dr. sc. Goran Delač

Izv. prof. dr. sc. Marin Šilić

Izv. prof. dr. sc. Klemo Vladimir

Faculty of Electrical Engineering and Computing
Consumer Computing Laboratory

Finding Frequent Itemsets

Izv. prof. dr. sc. Goran Delač

Copyright © 2019 CCL: Analysis of massive data sets

Outline

- **Motivation**
  - Market-Basket model
  - Applications
- **Finding frequent itemsets**
  - Computation model
  - A-Priori algorithm
  - Refinements

Copyright © 2019 CCL: Analysis of massive data sets

Motivation

Market-Basket Model



Copyright © 2019 CCL: Analysis of massive data sets

Market-Basket Model

Many-to-many relationship between ***items*** and ***baskets***



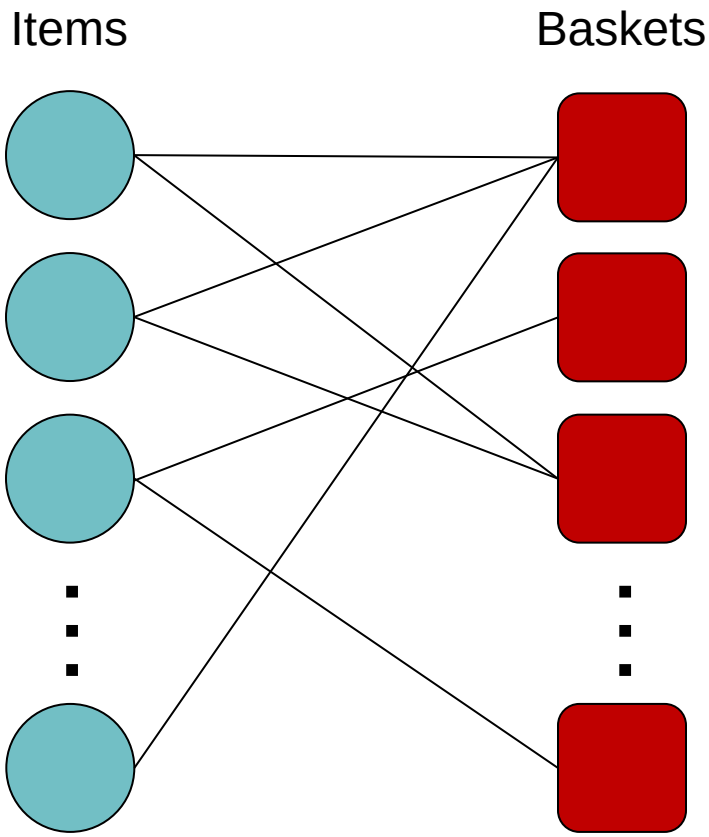
Problem: How to identify items that are ***frequently*** bought together?

Solution: Identification of association rules, i.e. discovery of frequent sets of items (baskets)

Copyright © 2019 CCL: Analysis of massive data sets

Market-Basket Model

Market-Basket Model: Many-to-many Relationship



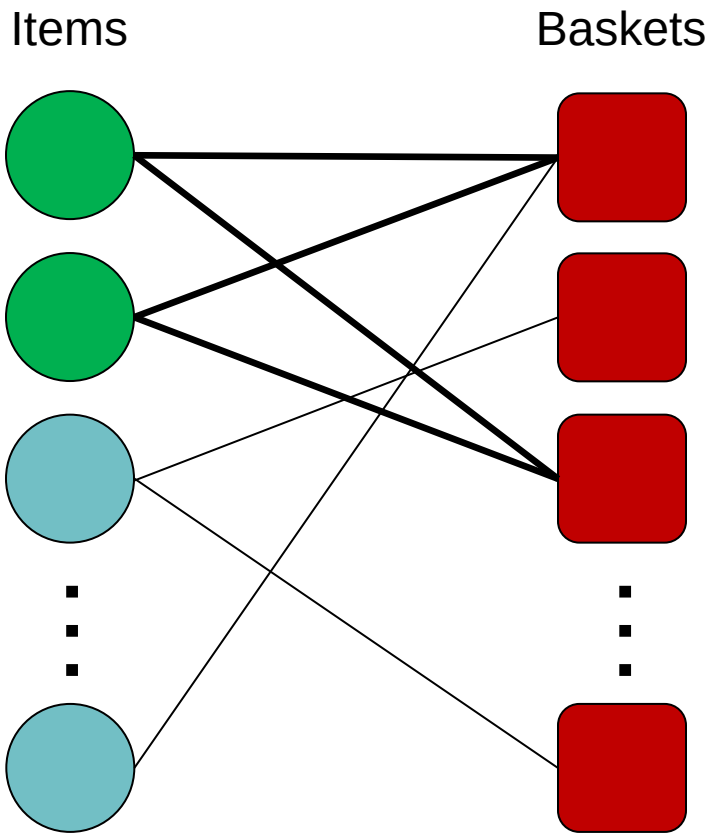
- Large number of items
- Large number of baskets
- Baskets contain a small subset of items

Problem: How many item pairs can be found in at least **k** baskets?

Copyright © 2019 CCL: Analysis of massive data sets

Market-Basket Model

Market-Basket Model: Many-to-many Relationship



- Large number of items
- Large number of baskets
- Baskets contain a small subset of items

Problem: How many item pairs can be found in at least **k** baskets?

k=2
The two green items are **often** bought together

Copyright © 2019 CCL: Analysis of massive data sets

Market-Basket Model

The story of beer and diaper

People with small children often buy beer and diapers together as they lack the time to go out

Rule: Put close together beer, diaper + … ? (chips,..)

Trick: Lower the price of beer, rise the price of diapers

Association rules need to be used ***frequently*** in order to be profitable

Copyright © 2019 CCL: Analysis of massive data sets

Frequent Itemsets vs. Similar Items

Frequent itemsets are about finding the ***absolute*** number a certain itemset appears (***frequency!***)

Finding similar items is about finding very ***similar***, but not necessarily the same sets of items (frequency is **not** important)

Copyright © 2019 CCL: Analysis of massive data sets

Frequent Itemsets vs. Similar Items

Frequent itemsets – traditional retailers (physical stores)

- Rules need to be frequent in order to be profitable – the store is a physical entity used by all customers

Finding similar items – web stores

- It pays off to know which items could be possibly bought together even by a small number of customers
- Finding correlation between rarely purchased items
- Profiting from the long tail

Copyright © 2019 CCL: Analysis of massive data sets

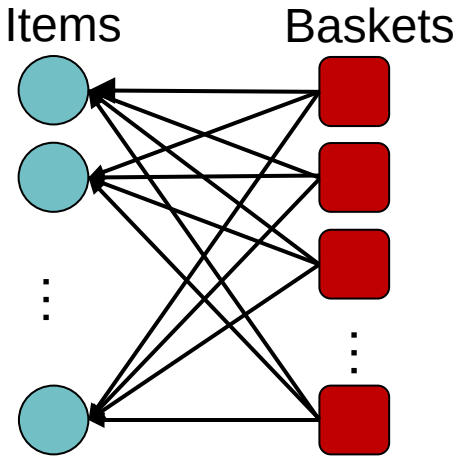
Other Applications

- **Any general many-to-many mapping between two kinds of entities**
- **Detecting drug side-effects**
  - Baskets: patients, Items: drugs and side-effects
- **Related concepts**
  - Baskets: documents, Items: words
    - Smaller documents: tweets, emails
    - Stop words are removed
- **Plagiarism**
  - Baskets: sentences, Items: documents

Copyright © 2019 CCL: Analysis of massive data sets

Other Applications

- **Biomarkers**
  - Baskets: patient information, Items: biomarkers and diseases
- **Finding Communities in Graphs (e.g. Facebook, Twitter)**
  - Baskets: nodes, Items: outgoing neighbors
  - Searching for **complete** bipartite subgraphs



Copyright © 2019 CCL: Analysis of massive data sets

Frequent Itemsets

- **Itemset Support**
  - ***Support*** for itemset ***i*** is the number of baskets that contain all items in ***i*** (in the observed dataset)
  - Expressed as an absolute number of baskets or more often as a fraction of the total number of baskets

- **Definition**
  - Given a support threshold ***s***, the item subsets that appear in at least ***s*** baskets are called ***frequent itemsets***

Copyright © 2019 CCL: Analysis of massive data sets

Frequent Itemsets

- **Items = {beer, chips, juice, wine}**
- **Support threshold = 3 / 6 baskets**

B_1 = {b, c, j, w}

B_2 = {b, c, j}

B_3 = {b, j, w}

B_4 = {c, j, w}

B_5 = {b, c}

B_6 = {b, w}

Frequent itemsets: {b}, {c}, {j}, {w}, {b,c}, {b,j}, {b,w}, {c,j}, {j,w}

Copyright © 2019 CCL: Analysis of massive data sets

Association Rules

- **Application of frequent itemsets!**
- **If-then rules that describe what a basket is likely to contain**
  - $\{i_1, i_2, i_3, \ldots i_n\} \rightarrow j$ implies that if a basket contains items $i_1, i_2, i_3, \ldots i_n$, it is also likely to contain item j
- **In practice, we want to find rules that really matter**
  - **Support**
  - **Confidence**
  - **Interest**

Copyright © 2019 CCL: Analysis of massive data sets

Association Rules

- **Association rule support**
  - **support($I \rightarrow j$) = support(I)**
- **Confidence**
  - For $I = \{i_1, i_2, i_3, \ldots i_n\}$
  - ***conf***$(I \rightarrow j)$ = support$(I \cup j)$ / support(I)
- **Not all high-confidence rules are interesting**
  - E.g. the rule $X \rightarrow$ *beer* can have high confidence if *beer* is simply purchased very often (independently of X)

Copyright © 2019 CCL: Analysis of massive data sets

Association Rules

- **Interest**
  - ***Fr* [*j*]** fraction of baskets containing item *j*
  - ***interest*(*I* → *j*) = *conf*(*I* → *j*) - *Fr* [*j*]**

B_1 = {b, c, j, w} B_4 = {c, j, w}

B_2 = {b, c, j} B_5 = {b, c}

B_3 = {b, j, w} B_6 = {b, w}

$c \rightarrow b$

conf ($c \rightarrow b$) = 3 / 4

Fr(b) = 5 / 6

interest($c \rightarrow b$) = 3/4 – 5/6

Copyright © 2019 CCL: Analysis of massive data sets

Association Rules

- Finding association rules
  - Find all association rules with
    - Support ≥ s (support of i_1, i_2, i_3, … i_n)
    - Confidence ≥ c
  - If a rule $\{i_1, i_2, i_3, \ldots i_n\} \rightarrow j$ has high support and confidence, both $\{i_1, i_2, \ldots i_n\}$ and $\{i_1, i_2, \ldots i_n, j\}$ will be ***frequent***
    - In realistic scenarios s ≥ 1%, c ≥ 50%
  - Interesting rules
    - Typically those with interest above 0.5 or below -0.5

Copyright © 2019 CCL: Analysis of massive data sets

Association Rules

- Finding association rules

1. Find all frequent itemsets ***I*** (taking into account ***s***)

2. Generate rules
   - For every subset ***A*** of ***I*** generate rule ***A*** → ***I*** \ ***A***
   - Compute **confidence** and select appropriate rules

Key problem: Finding frequent itemsets

Copyright © 2019 CCL: Analysis of massive data sets

Finding frequent itemsets

- Algorithms
  - A-priori
  - Elcat (Equivalence Class Transformation)
  - FP-growth (Frequent Pattern)

Copyright © 2019 CCL: Analysis of massive data sets

Finding frequent itemsets

- Computation model
- A-priori algorithm
- Refinements
  - Handling larger data sets in memory
  - Limiting the number of passes

Copyright © 2019 CCL: Analysis of massive data sets

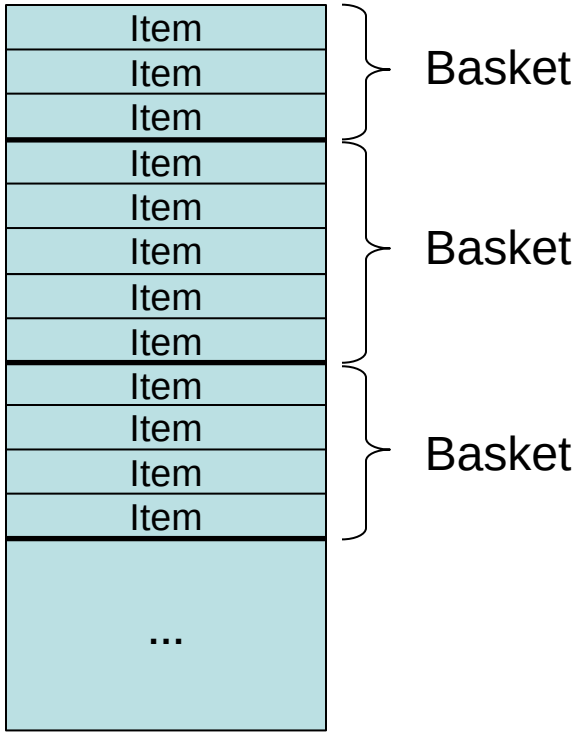
Finding frequent itemsets

- Naïve approach – Frequent pairs
  - Read the data once and count all the pairs
    - For each basket containing **n** items generate all the possible n(n-1) / 2 pairs in two nested loops
  - Issues: although **n** can be small, the overall number of baskets and items can be large
    - 368 million items sold by Amazon.com (Dec 2016)
    - That equals to $67 * 10^{15}$
    - If pairs counts are 4-byte integers, approximately 268 thousand terabytes are needed to store them!

Copyright © 2019 CCL: Analysis of massive data sets

Computation Model

- **Data Representation**
  - Data is usually stored in flat files
    - Stored on disk (or possibly DFS)
    - Sequences of baskets
    - Average basket size is small
  - The file does not fit into main memory
    - Many items and baskets
    - Baskets are expanded into pairs, triplets, …
    - I/O operations
    - Cost?



Copyright © 2019 CCL: Analysis of massive data sets

Computation Model

- **Cost of mining frequent itemsets**
  - Counting item subsets in each basket
    - **k** nested loops are used to generate subsets of size **k**
    - Usually, **k** is small and the baskets are small (contain **n** items), e.g. for basket size n = 30 and k = 2 there are $\binom{30}{2} = 435$ pairs
  - I/O operations - often dominant cost
    - Dataset often does not fit into main memory
    - Even if **k** gets large, **n** will decrease as there will be less baskets that meet the support threshold **s**, i.e. the itemsets in baskets are not considered frequent
    - Appropriate support threshold **s** needs to be selected

Copyright © 2019 CCL: Analysis of massive data sets

Computation Model

- **Main memory bottleneck**
  - Memory needs to be sufficiently large to store frequent itemset counts
  - Algorithms are **limited** by the memory size
    - Swapping the count data would diminish performance by many orders of magnitude

Copyright © 2019 CCL: Analysis of massive data sets

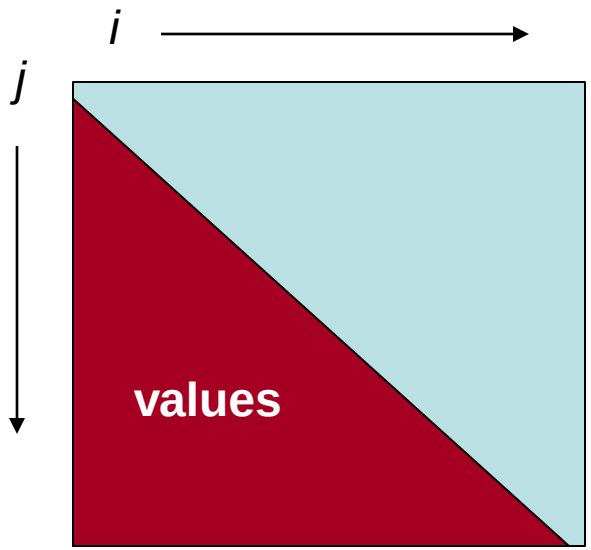
Computation Model

- **How to make it work for massive datasets?**
  - Limit the number of passes an algorithm makes over the data
  - Make frequent itemset counts fit into main memory

Copyright © 2019 CCL: Analysis of massive data sets

Computation Model

- **Storing counts in memory**
  - Item names should be indexed as integers (hash map)
  - Triangular matrix
    - 2-dimensional array $a[\, i, j \,]$, $i < j$



Copyright © 2019 CCL: Analysis of massive data sets

Computation Model

- **Storing counts in memory**
  - Triangular matrix
    - More generally one-dimensional array is used
    - Pair { *i*, *j* } is stored at *a*[*k*] for 1 ≤ *i* < *j* ≤ *n* where:

$$k = (i-1)\left(n - \frac{i}{2}\right) + j - i$$

    - If counts are stored as 4 byte integers, approximately $2n^2$ bytes are needed for *n* items

Copyright © 2019 CCL: Analysis of massive data sets

Computation Model

- **Storing counts in memory**
  - Hash table - Storing triplets
    - Triplet { *i*, *j, c* } is stored so that *i* < *j* are item indices and c is the itemset count
    - **Hash table** { *i*, *j* } → *c*
    - If counts are stored as 4 byte integers, 12 bytes are needed to store a singe itemset count
    - However, itemset counts are stored **only** when ***c* > 0**
    - More efficient if significantly fewer then **1/3** of the possible itemsets occur in the baskets (realistic scenario!)

Copyright © 2019 CCL: Analysis of massive data sets

Computation Model

- **Storing counts in memory**
  - Triangular matrix
  - Storing Triplets (hash table)
  - Still the use of main memory presents limitations as it might not be possible to store the data for all itemsets (pairs, triplets, etc..)
  - Can this situation be improved?

Copyright © 2019 CCL: Analysis of massive data sets

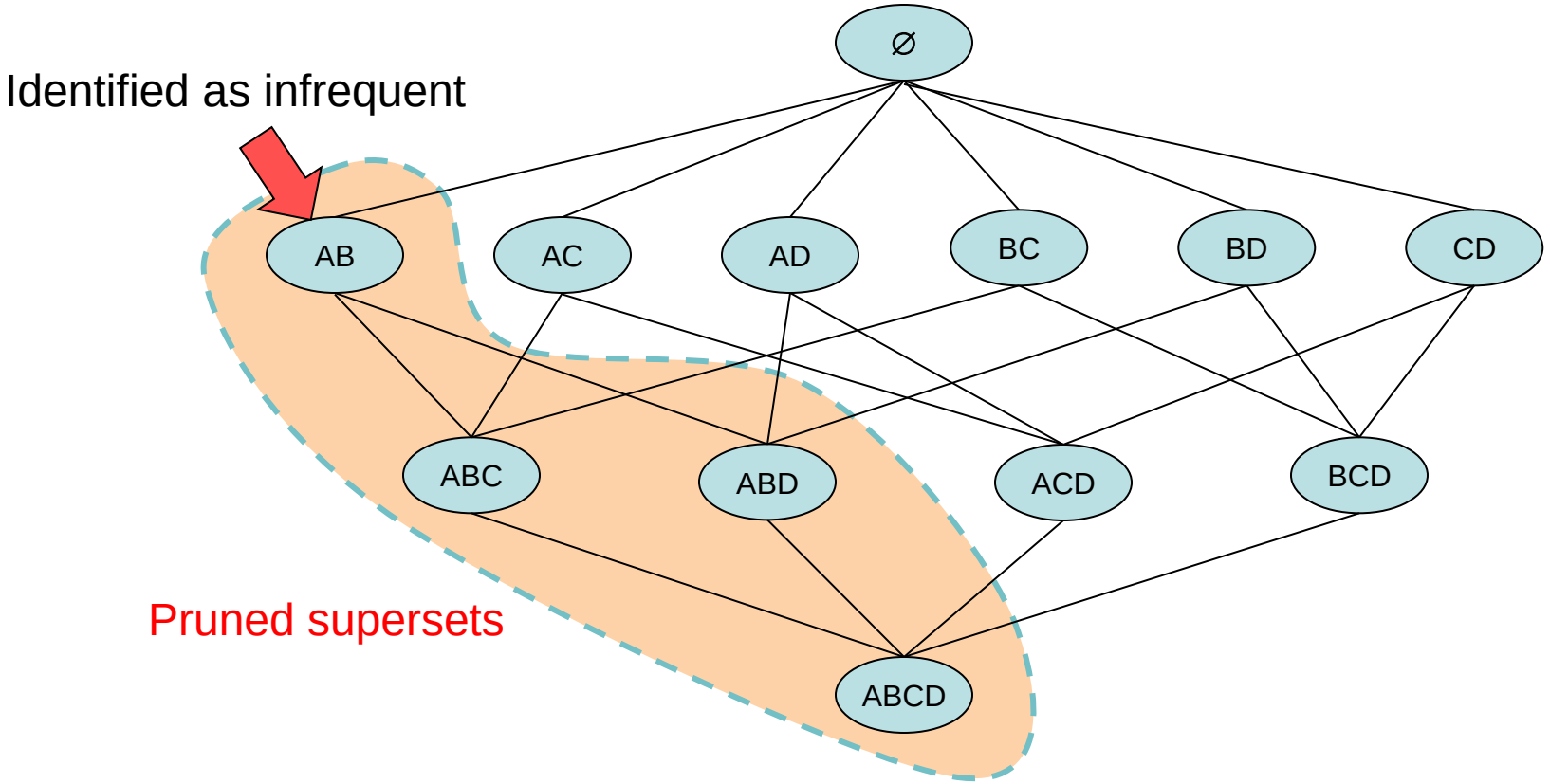
A-Priori Algorithm

- **Rakesh Agrawal (1993)**
- **2-pass algorithm that limits the need for main memory**
- **Key idea: Monotonicity of itemsets**
  - If an itemset ***I*** is **frequent** then each of its **subsets** must also be **frequent**
  - If ***I*** appears ***s*** times then, its subset ***J*** appears at least ***s*** times (it can also appear in other itemsets!)
  - If no superset of an itemset ***I*** is frequent, ***I*** is called the **maximal itemset**
- **Bottom-up approach**
  - start by counting pairs, then triplets, etc.

Copyright © 2019 CCL: Analysis of massive data sets

A-Priori Algorithm

- **Reducing problem space by pruning infrequent supersets**



Copyright © 2019 CCL: Analysis of massive data sets

A-Priori Algorithm

- **First pass**
  - Generate a table that translates item names to integers
    - Hash table
  - Count occurrence of each item across all the baskets
    - Array indexed by indices stored in the hash table
    - Size of the array is proportional to the number of items **n**
- **Intermediate step**
  - Generate frequent items table
    - **Scan** through the **counts** and mark the **items** that **meet** the support threshold **s** as **frequent**

Copyright © 2019 CCL: Analysis of massive data sets

A-Priori Algorithm

- **Second Pass**
  - For each basket look in the frequent items table and see which items are frequent
  - Generate all item pairs using **only** the **frequent items** in the basket
  - Add one to the count for each generated pair in the data structure used to store counts

Copyright © 2019 CCL: Analysis of massive data sets

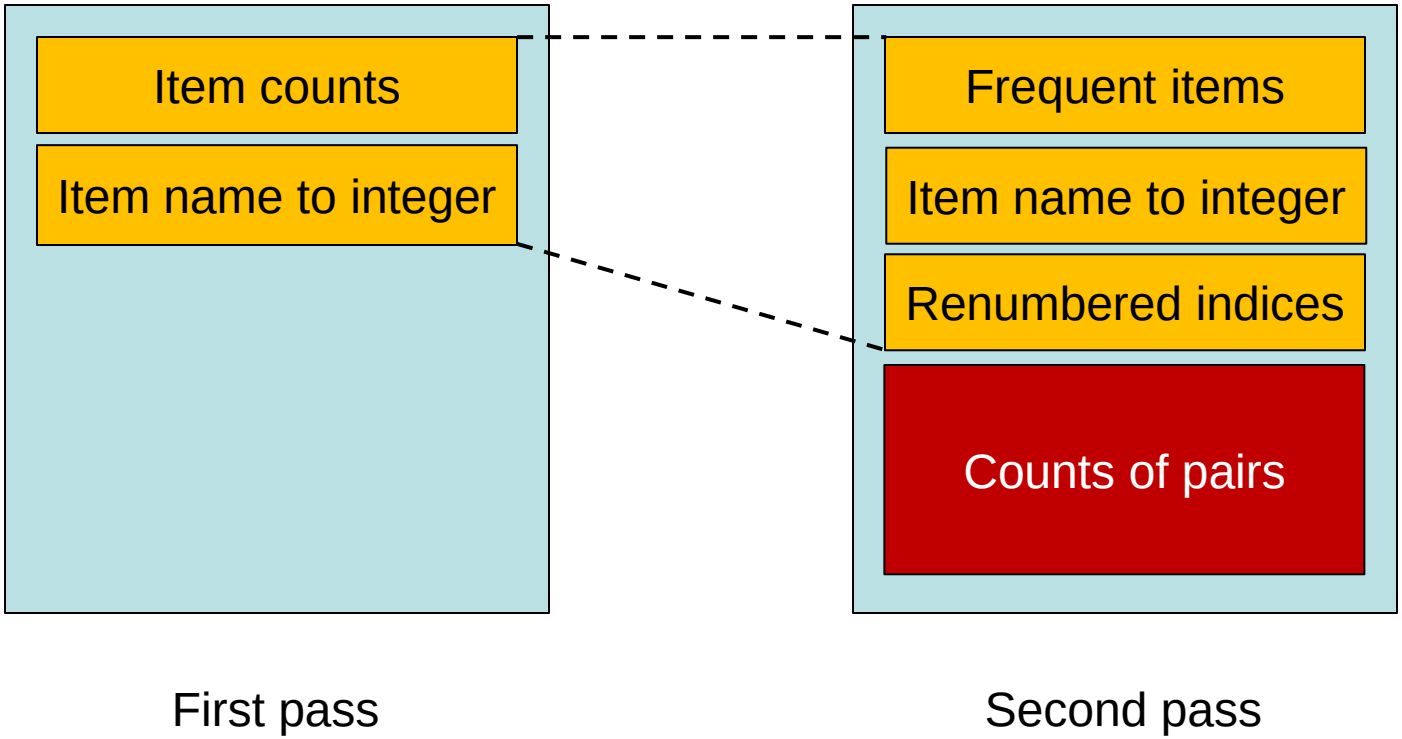
A-Priori Algorithm

- **Second Pass**
  - If the counts are stored in a triangular matrix, items should be renumbered to save space
  - A new table that translates the old indices is added
  - Then the storage size is $2m^2 < 2n^2$ where m is the number of frequent items and n total number of items

Copyright © 2019 CCL: Analysis of massive data sets

A-Priori Algorithm

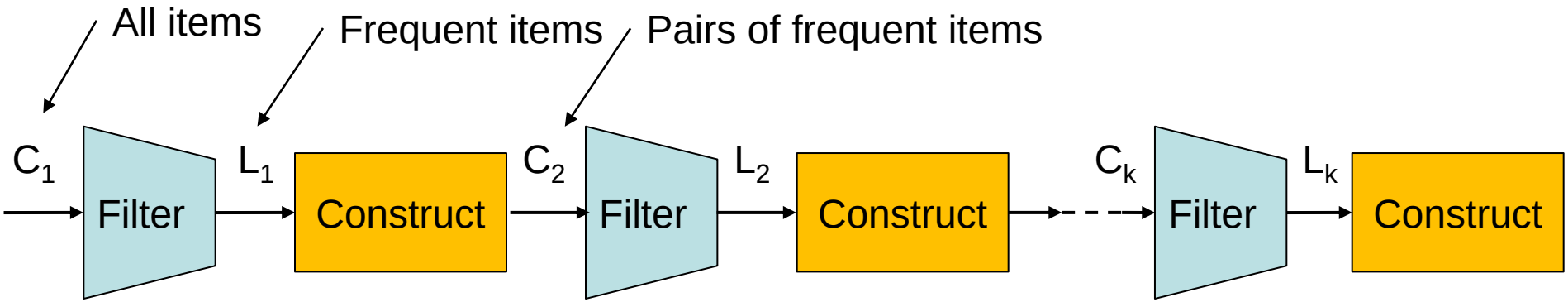
- **Main memory**



Copyright © 2019 CCL: Analysis of massive data sets

A-Priori Algorithm

- **All frequent itemsets**
  - **k-tuple**: repeat first and second pass ***k*** times
  - $\mathbf{C_k}$ – candidate tuples, have support ≥ s according to step k -1
  - $\mathbf{L_k}$ – filtered C_k so that only frequent k-tuples remain



Copyright © 2019 CCL: Analysis of massive data sets

A-Priori Algorithm

- **Beyond A-Priori**
  - Limitations on main memory
    - PCY algorithm
  - k passes through the data set are needed to find k-tuples
    - Random sampling
    - SON algorithm
    - Toivonen’s algorithm

Copyright © 2019 CCL: Analysis of massive data sets

PCY Algorithm

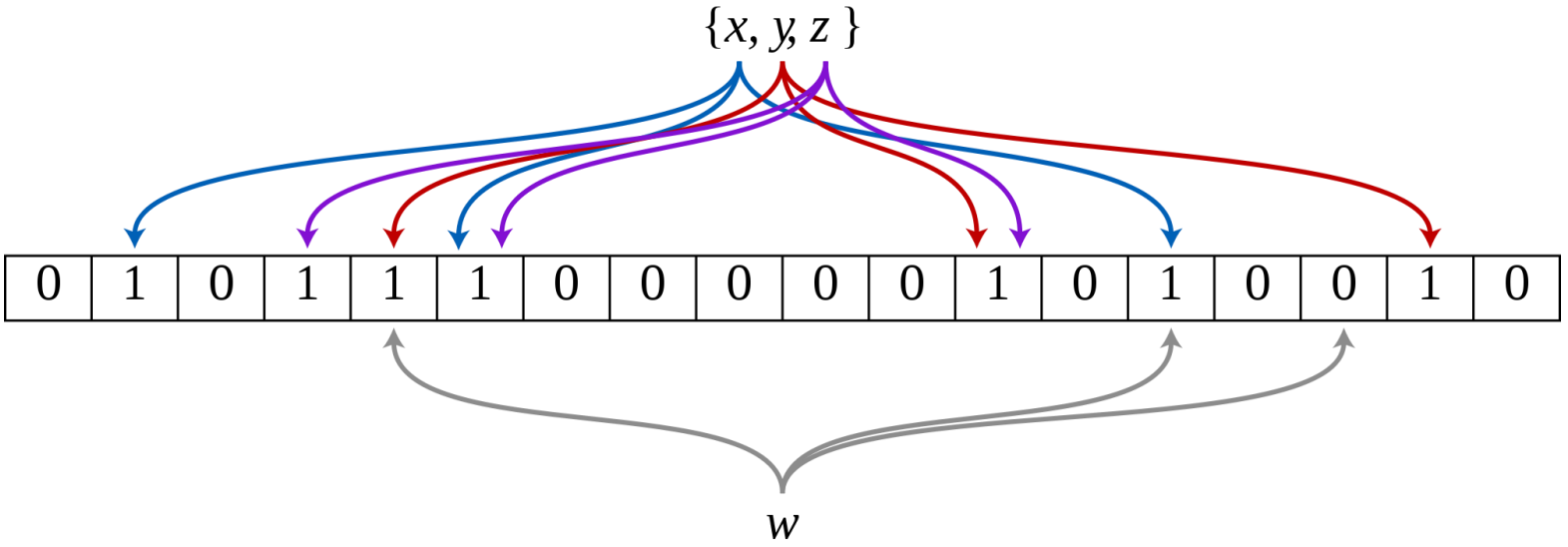
- **Park, Chen, Yu (1995)**
- **Improvement to A-Priori algorithm**

- **Idea**
  - **Most** of the main **memory** in the **first pass** of the A-Priori algorithm is **free**
  - This free space can be used to **reduce the memory requirements** in the second pass (storing itemset counts can be resource demanding)

Copyright © 2019 CCL: Analysis of massive data sets

PCY Algorithm

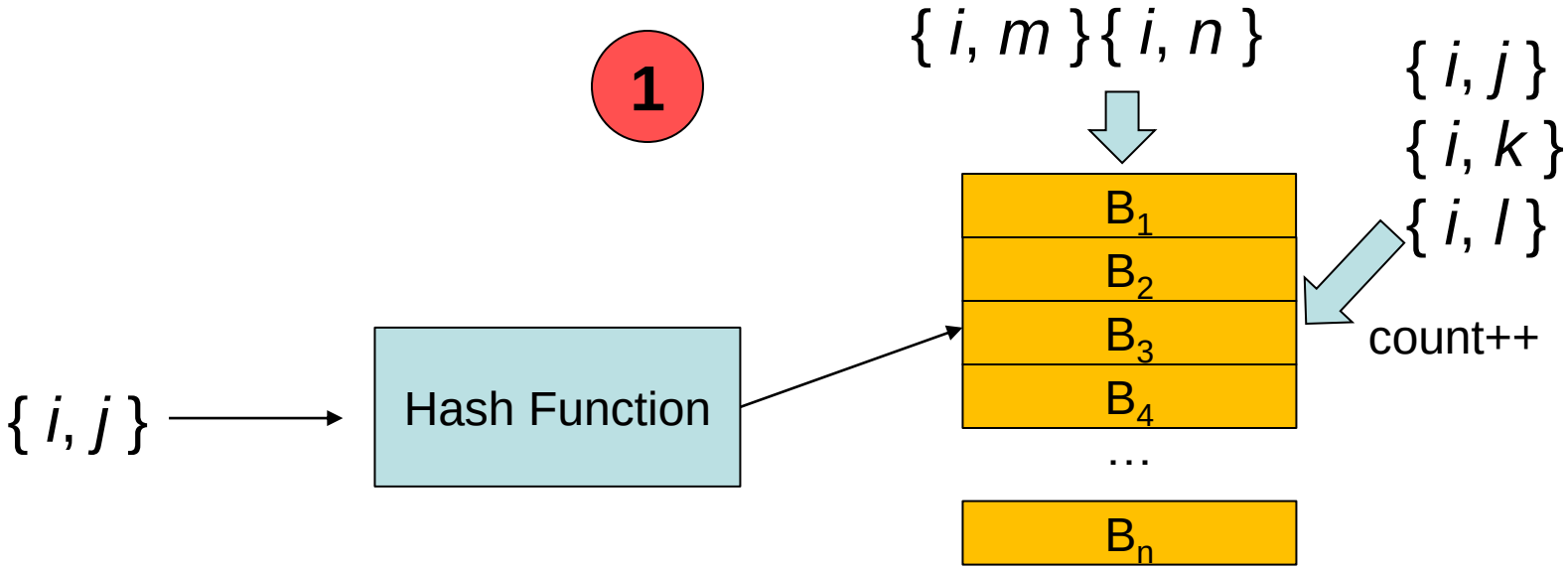
- **How to further reduce memory requirements?**
  - Approach similar to **Bloom filters**
    - Check if an element **is contained in a set**
    - Possible false positives
    - **No false negatives**



Copyright © 2019 CCL: Analysis of massive data sets

PCY Algorithm

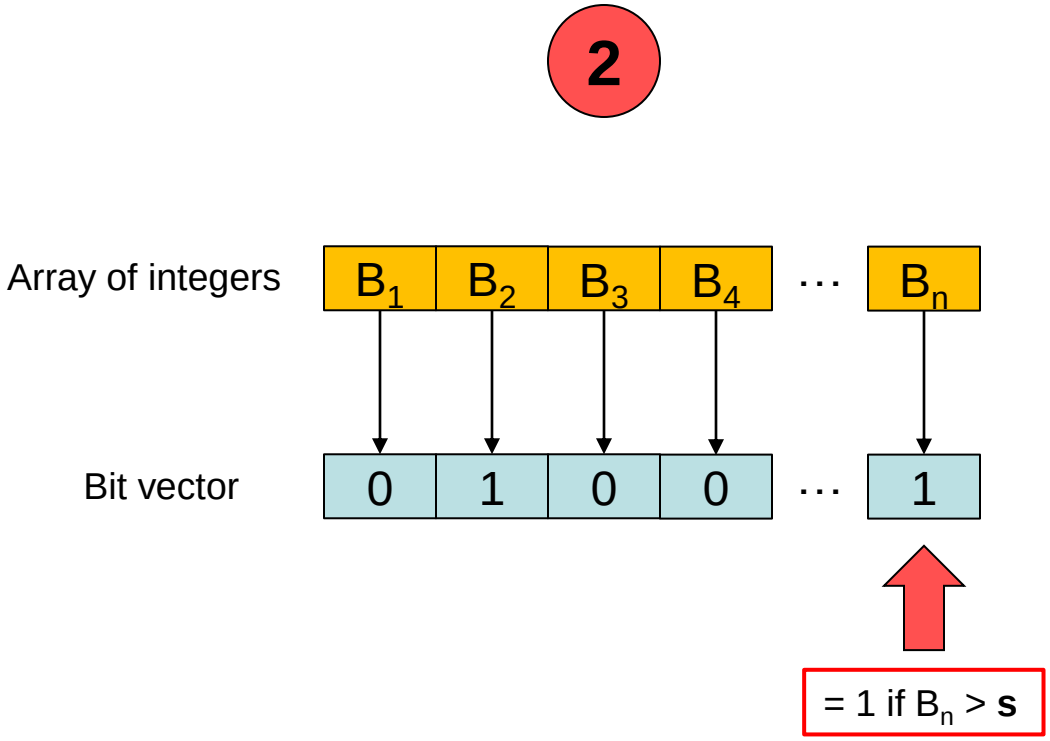
- **How to reduce memory requirements?**
  - Construct a **filter** for infrequent item pairs in the **first pass**



Copyright © 2019 CCL: Analysis of massive data sets

PCY Algorithm

- **How to reduce memory requirements?**
  - Construct a **filter** for infrequent item pairs in the **first pass**



Copyright © 2019 CCL: Analysis of massive data sets

PCY Algorithm

- **First pass**
  - Count items, and generate name to index translation table (A-priori)
  - For each **basket** create **item pairs** in a double loop
    - All item pairs in a basket are counted
    - Precision can be increased if only item pairs that consists of frequent items are considered (additional pass needed)
  - **Hash** each item pair into a **bucket** and increase its count
    - There can be as many buckets as the memory permits but crucially there can be less buckets than there are item pairs
    - Bucket counters do not require a lot of space (a few bytes). It is not necessary to count beyond the support threshold.

Copyright © 2019 CCL: Analysis of massive data sets

PCY Algorithm

- **Intermediate step**
  - Convert bucket counts into a bit-vector
    - Value 1 is assigned to **frequent buckets**, i.e. those that have count greater or equal to support **s**
    - Value 0 is assigned otherwise
  - 4-byte integer counts are replaced by 1bit, therefore a 1/32 of memory is needed to store the bit-vector

Copyright © 2019 CCL: Analysis of massive data sets

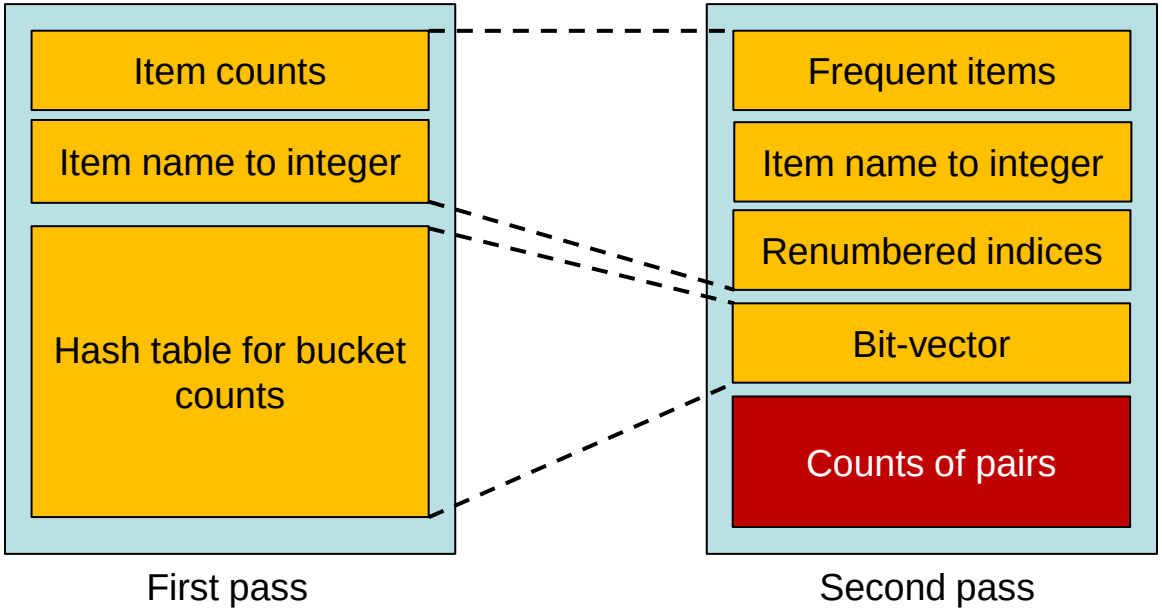
PCY Algorithm

- **Observation on frequent buckets**
  - If a bucket contains a frequent itemset, then it will have count larger or at least equal to **s**, i.e. it will be frequent
    - **However, it can contain 0 frequent itemsets and still be frequent!**
  - If a bucket has count less then **s**, then **none** of the itemsets that hash to it **are frequent**
    - **These itemsets do not need to be counted in the second pass**

Copyright © 2019 CCL: Analysis of massive data sets

PCY Algorithm

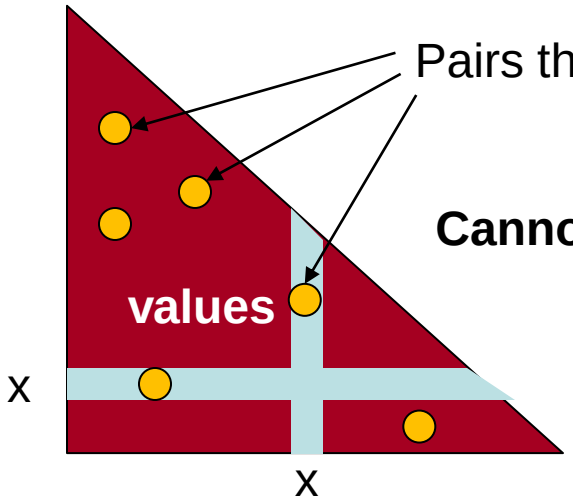
- **Second pass**
  - Count the item pair **{ *i*, *j* }** if and only if:
    - *i* and *j* are **both frequent**
    - The item pair { *i*, *j* } hashes to a bucket that has its bit-vector value set to **1**



Copyright © 2019 CCL: Analysis of massive data sets

PCY Algorithm

- **Remarks on the PCY algorithm**
  - It usually is more efficient but **not always**
  - Triangular matrix is not suitable to store counts
  - **Why?**
    - Pairs that can be skipped are randomly scattered
    - There is no known way to avoid unused spaces in the matrix



Cannot remove all entries for x!

Copyright © 2019 CCL: Analysis of massive data sets

PCY Algorithm

- **Remarks on the PCY algorithm**
  - Counts should be stored as triplets (e.g. Hash map)
    - PCY is **useful only** if it allows us to avoid counting at least **2/3** of the item pairs
    - Otherwise, A-Priori is more efficient
  - Memory-wise improvements to A-priori, like PCY are generally useful when it comes to finding frequent pairs
  - For triplets, … it is usually sufficient to use A-priori as memory requirements tend to be much lower

Copyright © 2019 CCL: Analysis of massive data sets

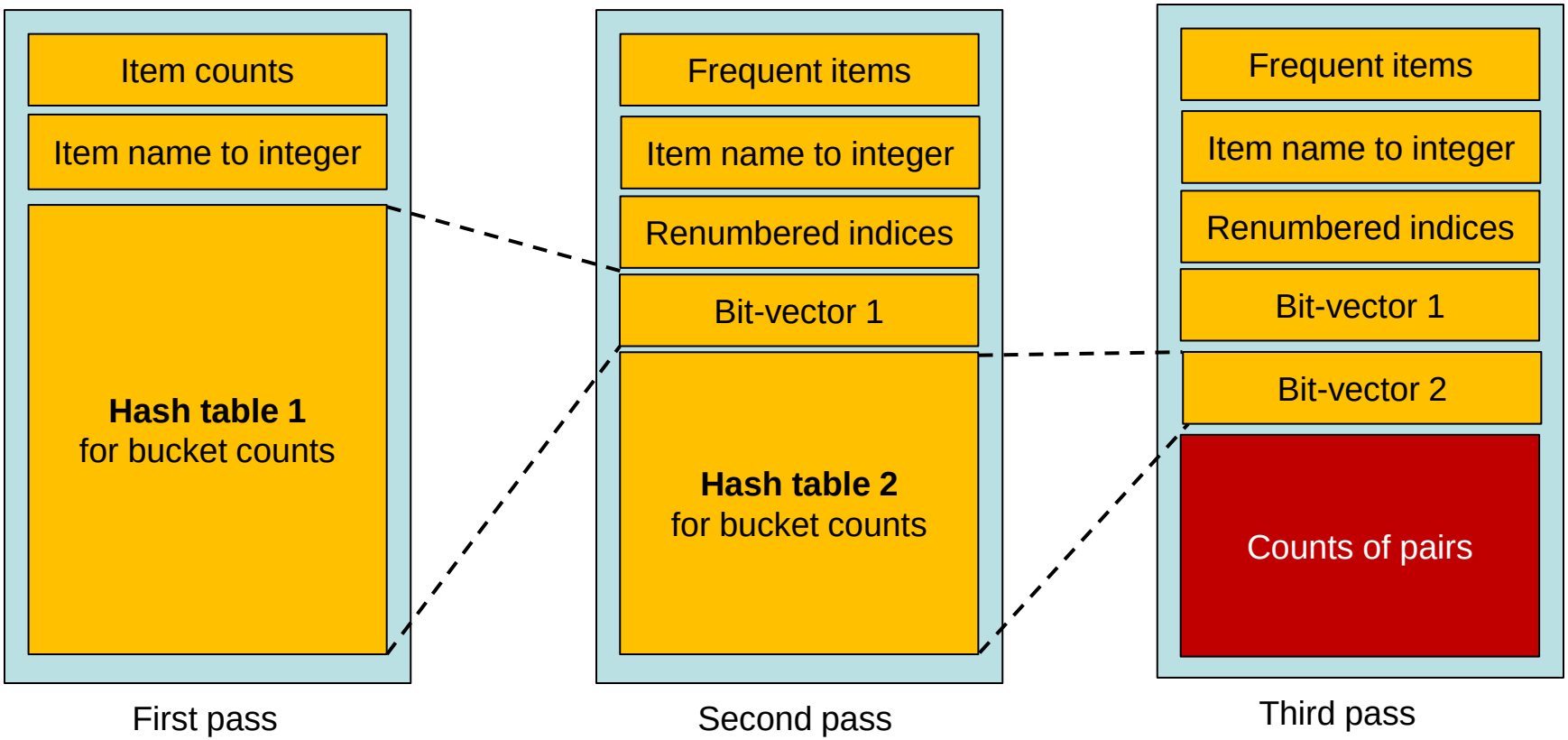
PCY Algorithm

- **Further improvements – Multistage Algorithms**
  - After first pass of PCY item pairs that are candidates to be frequent are **rehashed**
    - New hash function
    - Only pairs that hash to Bit-vector value **1** are counted (further reduces the number of false positives)
  - Number of candidate pairs is reduced, but an **additional pass** through the dataset is **needed**

Copyright © 2019 CCL: Analysis of massive data sets

PCY Algorithm

- **Further improvements – Multistage Algorithms**



Copyright © 2019 CCL: Analysis of massive data sets

PCY Algorithm

- **Further improvements – Multistage Algorithms**
  - Pair { *i*, *j* } is counted iff
    - Both i and j are frequent
    - { *i*, *j* } hash to bucket with **Bit-vector 1** value set to 1
    - { *i*, *j* } hash to bucket with **Bit-vector 2** value set to 1

Note: If { *i*, *j* } does not hash to 1 in bit-vector 1 this does not mean that it will not hash to 1 in bit-vector 2 (as other item pairs could have made this particular bucket in bit-vector2 frequent)

- **Thus it is crucial both bit-vectors are checked for each item pair**

Copyright © 2019 CCL: Analysis of massive data sets

Limiting the Number of Passes

- **Find frequent itemsets in ≤ 2 passes**
  - Fewer passes at the cost of not finding all frequent itemsets
    - This can be acceptable as having too many association rules is not useful
- **Random sampling**
- **SON algorithm**
- **Toivonen's algorithm**

Copyright © 2019 CCL: Analysis of massive data sets

Random Sampling

- **Randomly select a sample of the dataset**
  - Each entry is selected with probability **p**
    - Sample size is approx. **p** x **n**
  - If baskets are stored randomly
    - First p x n baskets are selected
    - Random DFS chunks are selected

- **Run A-priori or its improvements on the sample in the main memory**
  - Support threshold should be reduced proportionally
    - If sample size is 1% of the dataset s' should be approx. s/100 (the itemset need to appear in s/100 baskets)

Copyright © 2019 CCL: Analysis of massive data sets

Random Sampling

- **Not all frequent itemsets will be discovered**
  - **False positives**: itemsets frequent in the sample, but not in the dataset
  - **False negatives**: itemsets frequent in the data set, but not in the sample
  - However, if an itemset has support >> **s**, there is a low probability of it being a false negative

Copyright © 2019 CCL: Analysis of massive data sets

Random Sampling

- **False positives**
  - Can be eliminated by performing an additional pass in which their frequency is verified against **s**
    - Other itemsets do not need to be counted!

- **False negatives**
  - Can be reduced by further decreasing the support threshold and then eliminating false positives
  - Support threshold **s'** = 0.9 **p** x **n**
  - Lowering **s'** increases memory requirements as more items are counted

Copyright © 2019 CCL: Analysis of massive data sets

Toivonen's Algorithm

- **2 passes**
  - One on the data set sample
  - One on the whole data set
- **No false positives and false negatives!**
- **There is a small probability the algorithm will not produce an answer**
  - It can be repeated and the passes over the data set remain a small constant

Copyright © 2019 CCL: Analysis of massive data sets

Toivonen's Algorithm

- **Step 1**
  - Generate a random sample
  - If **p** is the portion of the original data set being sampled, **p x s** is the expected support threshold
  - The support threshold is selected to be lower than **p x s**
    - Usually 0.8 – 0.9 p s
    - This increases the memory demands but also reduces the probability that the algorithm will not produce an answer

Copyright © 2019 CCL: Analysis of massive data sets

Toivonen's Algorithm

- **Step 2**
  - Compute frequent itemsets in the sample
  - Construct the **negative border**
    - Collection of itemsets that are **not frequent** in the sample but **all** their subsets (constructed by removing a single item) **are frequent itemsets**
    - $\varnothing$ is considered to be frequent!

{b, c, j, w}

- Frequent: {b}, {c}, {j}, {b, c}, {b, j}
- Negative border: {w}, {c, j}

Copyright © 2019 CCL: Analysis of massive data sets

Toivonen's Algorithm

- **Step 3**
  - Pass through entire data set and count:
    - Itemsets frequent in the sample
    - Itemsets in the negative border
  - Outcomes
    1. **No itemsets in the negative border are frequent**
       - Then the itemsets that are confirmed to be frequent on the whole data set are the correct set of frequent itemsets
    2. **At least one itemset in the negative border is frequent**
       - No conclusion can be made – the algorithm needs to restart at Step 1

Copyright © 2019 CCL: Analysis of massive data sets

Toivonen's Algorithm

- **Why does this approach work?**
  - The main problem are false negatives
  - Assume that **S** is **frequent** in the data set but **not frequen**t in the **sample**
    - This would make it a FN candidate
  - Let **T** be a subset of **S**
    - Smallest subset not frequent in the sample

Copyright © 2019 CCL: Analysis of massive data sets

Toivonen's Algorithm

- **Why does this approach work?**
  - **T** must be in the negative border
    - If all its immediate subsets are not frequent, then T is not the smallest subset not frequent in sample
  - T is **frequen**t in the **data set**
    - By monotonicity – because S is frequent!
  - Thus, T is frequent and in the negative border
    - Toivonen yields no answer

Copyright © 2019 CCL: Analysis of massive data sets

SON Algorithm

- **Savasere, Omiecinski and Navathe (1995)**
- **Avoids false positives and negatives**
  - 2 full passes
- **Idea: Divide input into chunks**
  - Chunks are treated as samples
  - Apply random sampling algorithm with support **s'** set to: **p** x **s**, where **p** is chunk size (fraction of the original file)

Copyright © 2019 CCL: Analysis of massive data sets

SON Algorithm

- **Once chunks are processed**
  - Union of all itemsets that are frequent in at least one chunk
    - These are candidates for frequent itemsets
    - **Any** frequent itemset will **surely be frequent** in at least one chunk – no false negatives
- **Second pass:**
  - Count candidate itemsets and eliminate false positives

Copyright © 2019 CCL: Analysis of massive data sets

SON Algorithm: MapReduce

- **2 MapReduce passes**

- **First pass**
  - Map
    - **Input:** File chunk, size **p** x **n**, **p** – fraction of input file
    - Count item occurrences and select those with support > **p** x **s**
    - **Output**: **(F_i, 1)** key: F-frequent item, value: irrelevant
  - Reduce
    - **Input:** (F_1, 1), (F_2, 1), (F_3, 1), …, (F_{n-1}, 1), (F_n, 1)
    - **Output:** F_1, F_2, F_3, … , F_{n-1}, F_n

Copyright © 2019 CCL: Analysis of massive data sets

SON Algorithm: MapReduce

- **Second pass**
  - Map
    - **Input:** portion of input file + (F_1, F_2, F_3, … , F_{n-1}, F_n)
    - Count occurrences of frequent itemset candidates
    - **Output**: **(F_i, v)** key: F-frequent itemset, value: **v** count for **F_i** in the input file chunk
  - Reduce
    - **Input:** (F_1, v_1), (F_2, v_2), (F_3, v_3), …, (F_{n-1}, v_4), (F_n, v_n)
    - Select only those itemsets that have support > **s**
    - **Output:** F_i such that v_i, > **s,** for each **i**

Copyright © 2019 CCL: Analysis of massive data sets

Literature

1. J. Leskovec, A. Rajaraman, and J. D. Ullman, "Mining of Massive Datasets“, 2014, Chapter 6 : "Frequent Itemsets" (link)

2. R. Agrawal, and R. Srikant: "Fast Algorithms for Mining Association Rules in Large Databases “, VLDB '94 Proceedings of the 20th International Conference on Very Large Data Bases, Chile, 1994, pp. 487 – 499. (link)

3. A. Savasere, E. Omiecinski, and S. B. Navathe, "An Efficient Algorithm for Mining Association Rules in Large Databases“, VLDB '95 Proceedings of the 21th International Conference on Very Large Data Bases, Switzerland, 1995, pp. 432– 444. (link)

Copyright © 2019 CCL: Analysis of massive data sets